

Verge

AI Road-Risk & Near-Miss Intelligence

Finds dangerous junctions before the next crash: it detects near-misses and crashes in ordinary traffic video, and merges them with historical crash records into a transparent, ranked risk map for road-safety engineers.

LOW MODERATE HIGH CRITICAL

Problem statement PS-17 · Safe & Smart Communities · SDG 3 (Good Health) & SDG 11 (Sustainable Cities)
ACM Hackathon · Offline round · September 2026
Working prototype: runs fully offline on a laptop (FastAPI + plain HTML/JS, local ONNX models)

Contents

#	Section	What it covers
1	Problem statement	Why road-safety review needs leading indicators, not only crash counts
2	Proposed solution	What Verge does, how it addresses the problem, what is new
3	Architecture	System layers, data flow and components
4	Implementation	Tech stack, methodology, CV pipeline, conflict & crash logic, risk model
5	Working prototype	Screens from the running application
6	Results & validation	Measured before/after on real traffic clips, honest limits
7	Feasibility & viability	Cost, deployment, scalability, users
8	Problems faced & how we solved them	The real engineering challenges of the build
9	Risks & mitigation strategies	Technical, data, ethical and adoption risks
10	Impact	Who benefits and how
11	Future implementations	What this platform enables next
12	References	Methods, data and tools

Evaluation map

Evaluation point	Where to find it
Proposed solution (15)	Sections 1–2: explanation, how it addresses the problem, innovation and uniqueness
Implementation (10)	Sections 3–5: technologies, architecture, flowcharts, pipeline diagrams, prototype screens
Feasibility & viability (10)	Sections 6–9: validation, feasibility analysis, challenges, risks and strategies
Presentation & communication (10)	Problem framing (1), live demo flow (5), impact (10)
Documentation (5)	This report, repository README / architecture / benchmark docs, references (12)

SECTION 1

Problem statement

India records more road deaths than any other country. The Ministry of Road Transport & Highways reported **4,61,312 road accidents and 1,68,491 deaths in 2022**, and the WHO estimates about **1.19 million road deaths a year worldwide**. Most cities still decide where to intervene by looking at **past crash records**.

That approach has three structural problems:

- **It is reactive.** A junction only becomes a priority after people have already been killed or injured there.
- **Crash data is sparse and slow.** Serious crashes are rare events, records arrive months later, and many minor collisions are never reported, so the signal is thin.
- **Video is everywhere, but unused.** CCTV and dash-cam footage of dangerous interactions exists at many junctions, but nobody has the time to watch hours of it and turn it into evidence.

Road-safety research has an answer: **traffic conflicts** (near-misses) happen far more often than crashes and follow the same patterns, so they are a leading indicator of risk (the Swedish Traffic Conflict Technique, time-to-collision, post-encroachment time). What has been missing is a practical tool that measures them automatically from ordinary video and puts them next to the crash history, in a form an engineer can act on and defend.

"Which junctions in my city deserve road-safety attention first, and what is the evidence?"

That is the question Verge answers, for a municipal or traffic-police road-safety cell.

SECTION 2

Proposed solution

Verge is an offline, AI-assisted road-risk intelligence system with three parts that feed each other:

- **AI near-miss and crash detection from video.** Upload any traffic clip. Verge detects and tracks every road user (cars, buses, trucks, two-wheelers, bicycles, pedestrians), measures their motion on the road plane, and flags **near-misses** (time-to-collision, close passes) and **crashes** (observed collisions). Each event comes with an annotated evidence replay, a snapshot, plain-language reasons and a PDF evidence report.
- **A location-aware hotspot risk map.** Every monitored junction gets a transparent review-priority score that combines its **historical crash record** with **AI-observed evidence**. Scores are grouped into Low, Moderate, High and Critical, and are re-ranked the moment a video from that location is analysed.
- **Explain and share.** A ranked Recommendation Report with **View Analysis** for every location, a **text or audio summary** (English, Hindi, Marathi), a grounded **Road Safety Assistant** that answers questions only from Verge's own data, and **PDF reports that can be downloaded or emailed** in one click.

How it addresses the problem

Problem	How Verge addresses it
Reactive: waits for crashes	Near-misses are detected as they happen, so a junction can rise in priority before anyone is hurt.
Crash data is sparse / slow	Every analysed video adds measured evidence (count and severity of conflicts) to the site's score immediately.
Video is unused	Minutes of footage become a short list of events with replays and reasons; an engineer reviews seconds, not hours.
Decisions must be defensible	Every score splits into historical + AI-observed parts; every event shows TTC, distance, speeds and why it was flagged.
Privacy concerns with CCTV	Faces and heads are blurred before any frame is stored; the uploaded video is deleted; only anonymous trajectories are kept.

Problem	How Verge addresses it
Findings must reach decision makers	Plain-language summaries (also spoken in English / Hindi / Marathi), a question-answering assistant and one-click emailed PDF reports.

Innovation and uniqueness

- **Crash and near-miss in one engine.** Most tools do either crash detection or counting. Verge measures the whole spectrum: approach → near-miss (TTC, close pass) → crash (contact + impact signature), with severity.
- **Calibration-free geometry.** Verge fits a **perspective ground-plane model from the vehicles themselves** (their apparent size grows linearly with image row), classifies the view (near-overhead CCTV vs street level), and measures street-level clips in a **bird's-eye road plane**. No camera calibration, no manual setup.
- **Evidence, not a black box.** Every flag lists the measured values that triggered it, and the map score is a published weighted sum. A judge or engineer can trace data → score → risk level → map colour.
- **Honest by design.** When a clip cannot be measured (handheld or zooming camera), Verge says "near-misses not assessed" instead of guessing. It never invents counts or locations.
- **A grounded assistant, not a chatbot.** The Road Safety Assistant uses no language model: every answer is assembled from the same hotspot records, analysis results and settings the rest of the app uses, and it says "I don't have that evidence" when the data can't answer. One source of truth means the map, report, summary, assistant and PDF can never disagree on a number.
- **Fully offline and low cost.** Runs on one laptop (GPU optional) with local models and a vendored map library, so it can be demonstrated and deployed without cloud services or data leaving the office.

SECTION 3

Architecture

Verge is a three-layer application. A plain HTML/JavaScript front end talks to a FastAPI back end over REST; all models and data are local files. Nothing requires an internet connection except the optional OpenStreetMap background tiles.

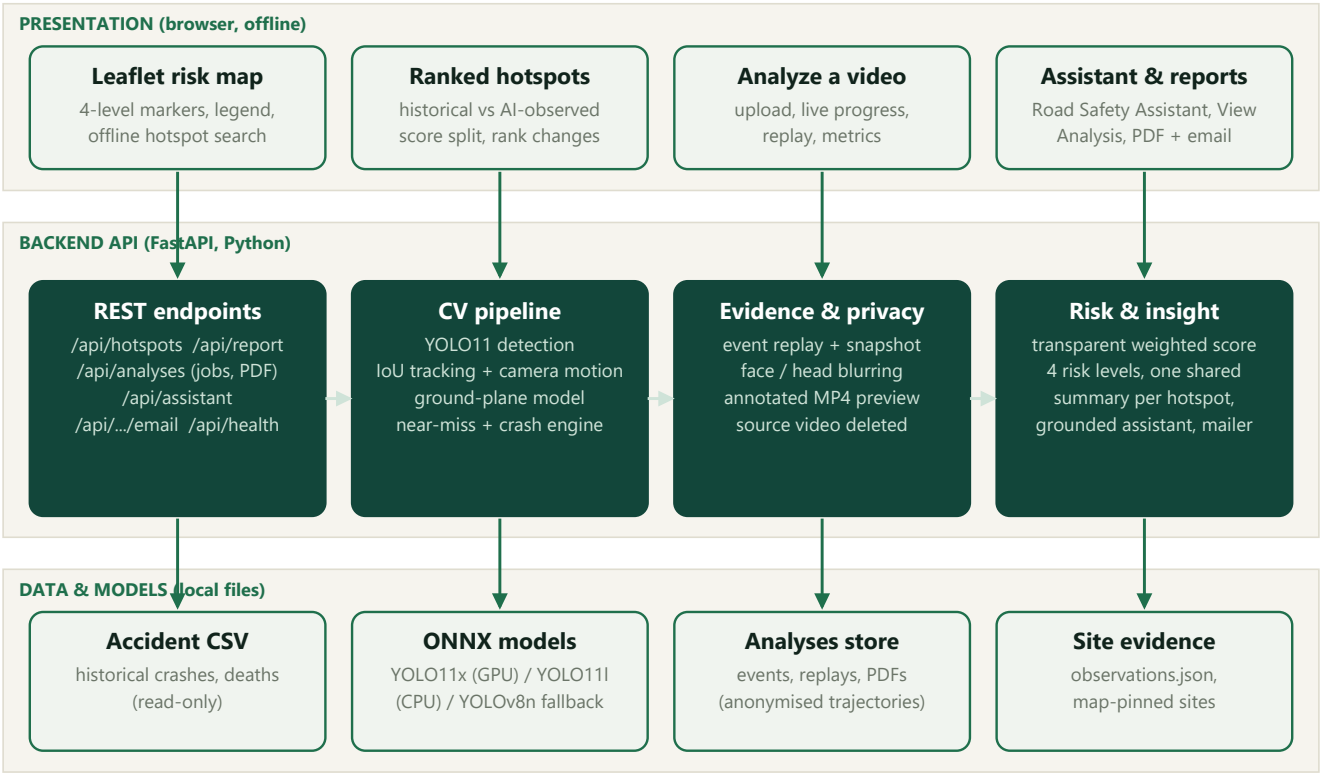


Figure 1 — System architecture. Arrows show the request and data flow.

Component responsibilities

Module	Responsibility
detection_yolo.py	YOLO11 inference (ONNX Runtime DirectML on GPU, OpenCV DNN on CPU), class filtering, rider/occupant removal, camera-motion estimation
tracking.py	IoU tracker with constant-velocity prediction, confirmation after 4 hits, duplicate suppression, majority-vote labels
conflict_engine.py	Per-clip perspective fit, view classification, bird's-eye mapping, TTC / close-pass / crash rules, event lifecycle
video_analysis.py	Orchestrates a clip: detection → tracking → conflicts → replay, snapshot, annotated preview, explanations
privacy.py	Face (Haar) and pedestrian-head blurring on every stored frame
risk_scoring.py / site_evidence.py	Weighted review-priority score, historical vs AI-observed split, risk tiers, rank changes
evidence_report.py / pdf.py	Dependency-free PDF evidence report per event
hotspot_insight.py / site_report.py	One shared structured summary per hotspot (used by View Analysis, audio, assistant, PDF); site review brief PDF
assistant.py	Road Safety Assistant: intent matching over hotspot data, the latest analysis and the implementation's own settings
mailer.py / envfile.py	Server-side SMTP delivery of the same PDF as the download; credentials from environment / .env; test outbox mode
main.py / analysis_service.py	REST API, background jobs with real stage-by-stage progress, friendly errors
frontend/ (app.js, index.html, style.css)	Leaflet map, hotspot search, ranked list, report table + View Analysis, summaries (Web Speech API), assistant panel, email dialog

SECTION 4

Implementation

Technology stack

Layer	Technology	Why this choice
Language	Python 3.13, JavaScript (ES2020)	Python for CV and data; plain JS so the UI needs no build step
Object detection	YOLO11x / YOLO11l (Ultralytics, COCO), exported to ONNX; YOLOv8n fallback	State-of-the-art accuracy; ONNX runs without PyTorch. YOLO11x found ~86% of hand-counted vehicles on our CCTV test vs ~17% for YOLOv8n
Inference	ONNX Runtime + DirectML (GPU), OpenCV DNN (CPU)	Any DirectX 12 GPU on Windows, no CUDA install; automatic CPU fallback
Computer vision	OpenCV 4 (optical flow, affine estimation, Haar cascades, video I/O), NumPy	Camera-motion compensation, privacy blurring, rendering
Back end	FastAPI + Uvicorn	Async REST API, background analysis jobs, auto docs
Front end	HTML5, CSS3, JavaScript, Leaflet (vendored), OpenStreetMap tiles	Lightweight, offline-capable interactive map
Reports	Custom PDF writer (JPEG + text), Markdown report	No heavy dependencies; evidence PDFs in under a second
Assistant	Deterministic intent engine over Verge data (Python), no LLM	Cannot hallucinate numbers; works fully offline
Speech	Browser Web Speech API (SpeechSynthesis)	Audio summaries in English, Hindi, Marathi with no cloud service

Layer	Technology	Why this choice
Email	Python smtplib (SMTP / STARTTLS), settings in .env	No extra package; credentials stay on the server
Testing	pytest (55 tests), Playwright end-to-end browser tests	Regression safety for every algorithm and UI change
Hardware	Laptop CPU; optional GPU (tested on NVIDIA RTX 4060)	GPU: an 8 s clip is analysed end to end in ~11 s

Methodology: the video pipeline

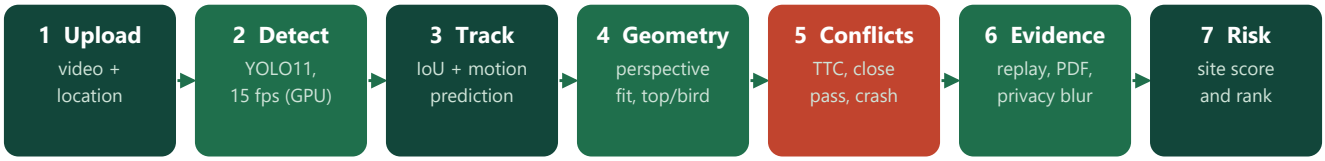


Figure 2 — From an uploaded clip to an updated risk map.



Figure 2b — The decision-support workflow: the same hotspot context flows through every step.

- **Detection.** Frames are sampled at 15 fps (GPU) or 10 fps (CPU) and passed to YOLO11. Only road-user classes are kept; riders and vehicle occupants are merged into their vehicle; part-of-vehicle boxes are removed.
- **Tracking.** An IoU tracker with motion prediction links detections into tracks, confirms a track after 4 hits and keeps it through 1.5 s of missed detections. Camera shake is removed with optical-flow affine estimation.
- **Geometry.** From the clip's own vehicles, Verge fits scale = a·(row − horizon). A horizon far above the frame means a near-overhead CCTV view (box footprints used directly); otherwise each road user is mapped to road-plane metres (Z = f/s, X = (u − cx)/s) and given a vehicle-sized footprint oriented along its motion.
- **Conflicts.** For every pair of road users in every frame: gap, closing speed and time-to-collision; then the near-miss and crash rules below, persistence checks and one event per interaction.
- **Evidence.** Each event gets a replay cut from its real start, peak and end, a peak snapshot, reasons, PDF, and privacy blurring. The site's score and rank are then recomputed.

Conflict and crash logic

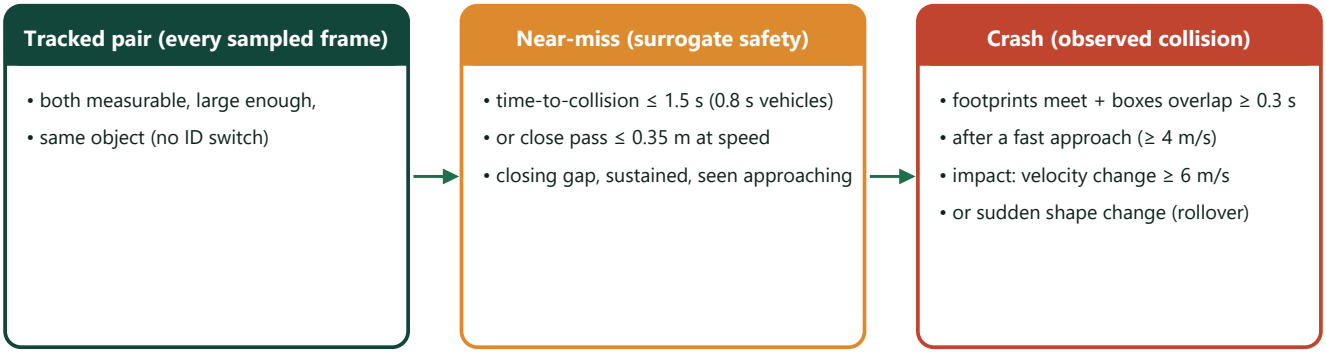


Figure 3 — Every flag must pass all conditions in its column; thresholds live in config.py.

Crash detection follows the approach used in published CV accident-detection work (overlap of road users followed by an acceleration or trajectory anomaly, e.g. Ijjina et al., 2019), made robust for monocular video: the impact must be a sustained change measured on both sides of the contact, and both tracks must stay on the same objects, so tracker ID switches in dense traffic are not mistaken for impacts.

Risk model: from data to map colour

Review priority (0–1) = 0.35 × crashes + 0.30 × fatalities + 0.25 × AI conflict events + 0.10 × conflict severity, each factor min-max normalised across monitored sites. A crash detected by the AI counts as a conflict event at maximum severity (1.00).



Figure 4 — Risk levels on the same score. Crash history alone can reach 0.65, so Critical (≥ 0.60) is reached by sites at or near the worst crash and fatality record, or with strong AI-observed evidence.

SECTION 5

Working prototype

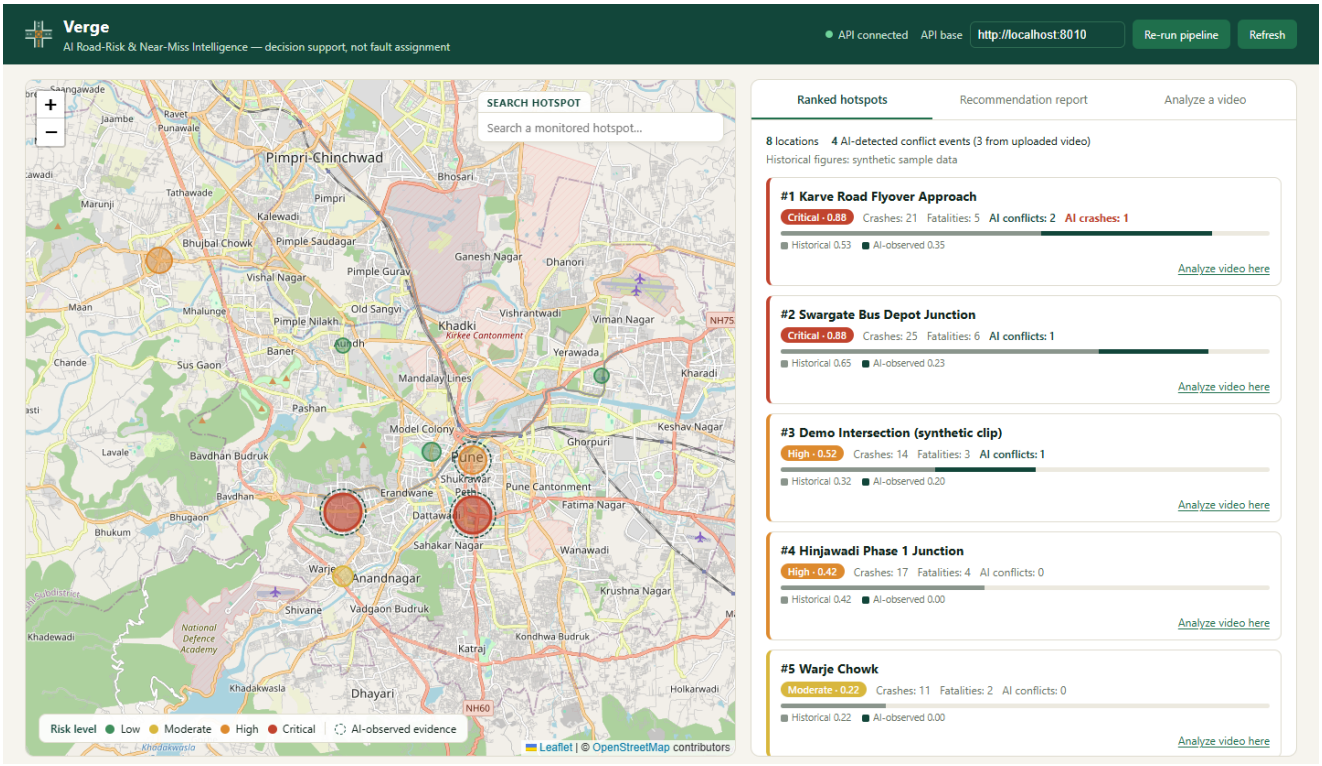


Figure 5 — Risk map with four levels, legend, hotspot search and the ranked list, after a crash video was analysed at Karve Road (now #1, Critical). Swargate is Critical on its crash history alone.

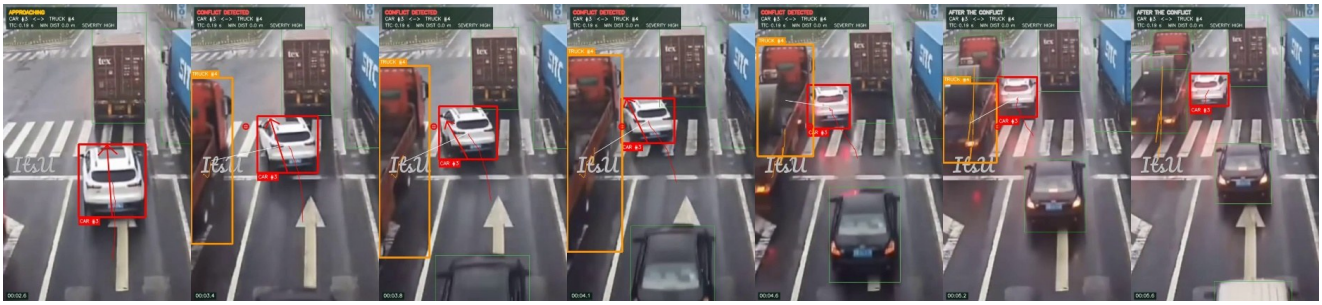


Figure 6 — AI conflict replay (CCTV clip): the HUD moves from APPROACHING to CONFLICT DETECTED; the marker shows where the truck squeezes the car.

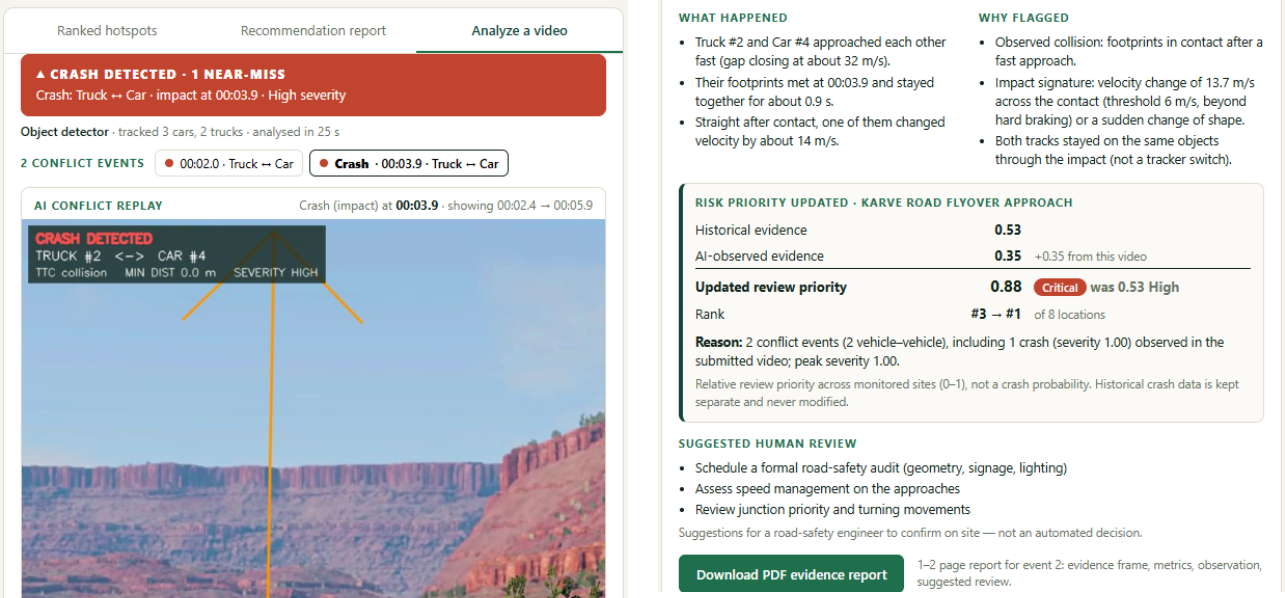


Figure 7 — A crash clip: "Crash detected · 1 near-miss", the measured evidence behind the flag, and the location-linked risk update (Karve Road 0.53 High → 0.88 Critical, rank #3 → #1).

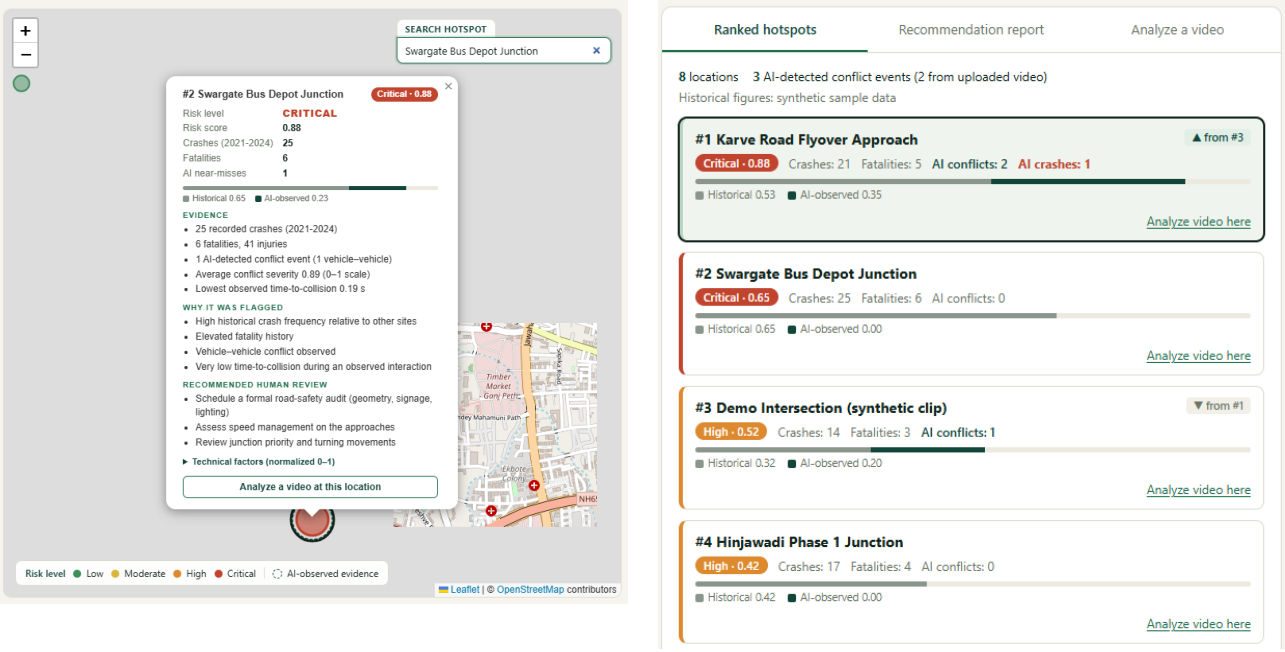


Figure 8 — Offline hotspot search ("swargate" → map centred, marker ringed, details opened) and the re-ranked hotspot list after a video analysis, with AI-observed crashes counted.

#	Location	Priority	Crashes	Fatalities	AI conflicts	
1	Swargate Bus Depot Junction	Critical · 0.65	25	6	0	View Analysis
2	Karve Road Flyover Approach	High · 0.53	21	5	0	View Analysis
3	Demo Intersection (synthetic clip)	High · 0.46	14	3	1	View Analysis
4	Hinjawadi Phase 1 Junction	High · 0.42	17	4	0	View Analysis
5	Pinned location (18.5560, 73.8612)	Moderate · 0.35	n/a	n/a	4	View Analysis
6	Warje Chowk	Moderate · 0.22	11	2	0	View Analysis
7	FC Road x JM Road Junction	Low · 0.13	9	1	0	View Analysis
8	Nagar Road Signal	Low · 0.03	6	0	0	View Analysis
9	Aundh Road Crossing	Low · 0.00	4	0	0	View Analysis

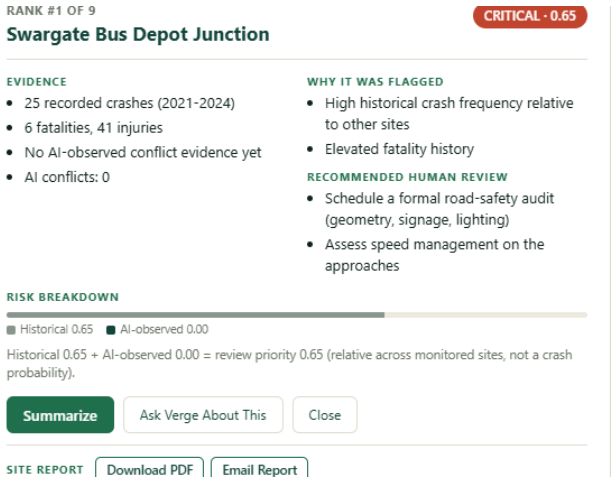


Figure 9 — Recommendation Report: the ranked table with View Analysis per location, and the analysis dialog (evidence, why flagged, recommended review, risk breakdown, Summarize, Ask Verge About This, site PDF and Email Report).

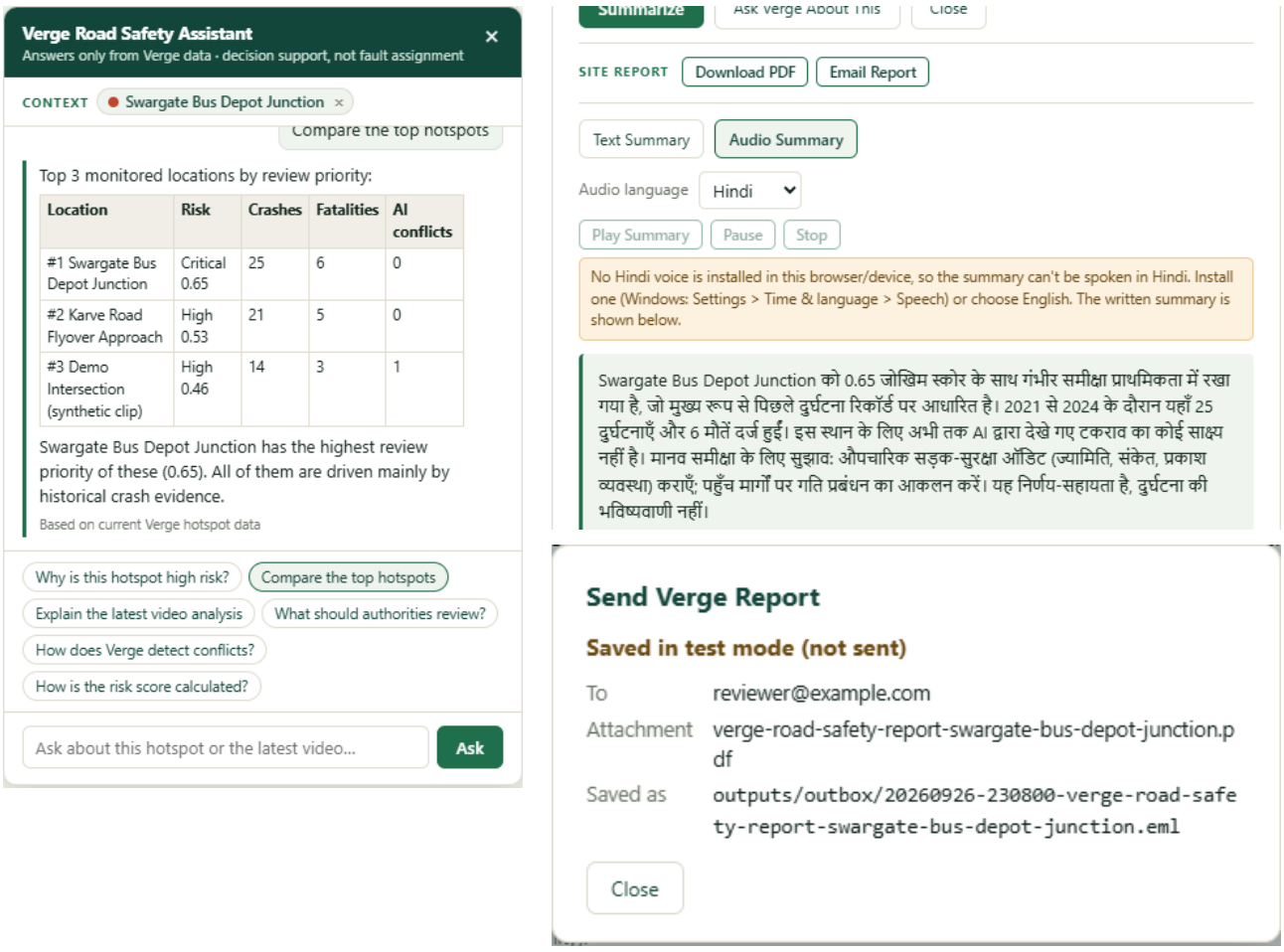


Figure 10 — Verge Road Safety Assistant with the hotspot context chip and a data-driven comparison; the audio summary in Hindi (with an honest message when no Hindi voice is installed); and an emailed report (test mode shown: saved, clearly not claimed as sent).

Demo flow

- Open Verge: the map immediately shows which junctions deserve the most review (four risk levels).
- Search a junction (e.g. Swargate): the map centres on it and shows its crashes, fatalities and score.
- View Analysis → Summarize (text or audio) → Ask Verge About This for follow-up questions.
- Upload a traffic video for a location: near-miss or crash detected, evidence replay, metrics and reasons.
- The site's review priority and rank update on the map; download or email the PDF report.

SECTION 6

Results & validation

We validated on 15 clips: real CCTV and street footage (WhatsApp CCTV clips, Pexels junction videos, Indian junction clips) and one simulated crash. There is no labelled ground truth for these clips, so every reported event was checked frame by frame; the figures below are counts and timings, not a statistical accuracy claim.

Measure	First prototype (motion-based)	Current Verge
Indian junction clip (39 s): tracked objects	1,190 (1,107 fake "pedestrians")	real road users only (≤ 20 visible at once)
Same clip: reported near-misses	232	0 false events
Dense scooter clip: reported near-misses	2,046	2 (flagged for human review)
All 14 real clips: reported events	36 (after the first fix)	9 near-misses: 4 confirmed or credible, 5 flagged for human review
CCTV truck–car squeeze (2 recordings)	missed / wrong pair	detected, correct pair, both recordings
Simulated crash clip	nothing detected	1 crash + 1 near-miss, correct vehicles and times
Handheld / drone clips	false conflicts	"not assessed", with the reason shown
Analysis time	210 s for a 39 s 720p clip	~38 s for a 37 s clip, ~11 s for an 8 s CCTV clip (GPU, YOLO11x)
Road Safety Assistant	—	answers from exact data; declines unavailable questions (23/23 browser checks)
Email Report	—	same PDF as the download attached; honest "not configured" / test-mode states
Automated tests	—	55 unit tests + end-to-end browser tests passing

Replay, snapshot, metrics and PDF were checked programmatically for every event: each describes the same moment and pair. On the 15 clips, the crash rule reports exactly one crash: the real one.

Known limits (stated in the app and the documentation): distances come from one uncalibrated camera and are approximate; very dense traffic can still produce borderline events that need a human look; night and aerial footage are harder for the detector; the historical crash data in the prototype is a synthetic sample in the MoRTH format.

SECTION 7

Feasibility & viability

Aspect	Analysis
Technical	Built and working end to end on a single laptop; open, mature components (YOLO, OpenCV, FastAPI, Leaflet). No calibration step: the geometry is learned per clip.
Cost	Uses existing CCTV / dash-cam footage and a commodity PC. No cloud fees, no special sensors. GPU optional.
Deployment	Offline by default: runs inside a municipal or police network, data never leaves the office. One-click start script on Windows; plain browser UI.
Scalability	Stateless analysis jobs: several machines can process footage in parallel; the site model already handles any number of locations (CSV sites plus map-pinned sites).
Data	Designed for the MoRTH / data.gov.in accident format; swapping the sample CSV for the real dataset is a file change.
Users & value	Road-safety cells, traffic police, smart-city command centres, researchers. Value: earlier, evidence-based prioritisation of limited engineering budgets.

Aspect	Analysis
Licensing	YOLO models are AGPL-3.0 (Ultralytics; commercial licence available); other components MIT / Apache-2.0 / BSD.

SECTION 8

Problems faced & how we solved them

Problem we hit	What was happening	How we solved it
Thousands of fake road users	Background subtraction turned every moving truck into many blobs labelled "pedestrian".	Replaced with a trained YOLO detector; kept the motion detector only for synthetic demo clips.
One vehicle, dozens of IDs	Tracks broke whenever a detection was missed or boxes overlapped.	IoU tracker with motion prediction, confirmation after 4 hits, 1.5 s memory, duplicate suppression.
Fake near-misses from depth	In street-level views, cars a lane apart or metres behind each other touch on screen.	Per-clip perspective fit + bird's-eye road-plane measurement; view classification (overhead vs oblique).
Missed truck–car squeeze	From above, a truck's box is its ground footprint and a cut-off truck got the wrong size.	Overhead footprints use most of the box; scale for cut-off vehicles comes from the ground model.
Tracker ID switches	In dense traffic the box hops to another vehicle, which looks like a sudden jump or impact.	Jump detection with depth-aware tolerance; crashes need the same object to be kept through the impact.
No crash detection	A crash is exactly what the noise filters reject: merging boxes, sudden motion, shape change.	Dedicated crash rule: contact after a fast approach + sustained velocity or shape change (rollover).
Portrait phone videos	Size and focal assumptions were tied to the frame height.	Size thresholds use the shorter side; focal length uses the long side.
Handheld / drone footage	A moving camera makes every distance meaningless.	Camera-motion estimate; above a travel/zoom limit the app says "not assessed" instead of guessing.
15-minute analysis times	Full-resolution encoding and face detection on every frame.	GPU inference (DirectML), decode only output frames, JPEG preview sequence, blur on a reduced copy.
Critical hotspots never shown	History can score at most 0.65, but Critical started at 0.75.	Re-derived the bands from the score's real range (0.20 / 0.40 / 0.60); data unchanged.
Chatbot hallucination risk	A general chatbot can invent crash numbers or causes, which is unacceptable for evidence.	Grounded, deterministic assistant over Verge data only; explicit "not available" answers.
Numbers disagreeing across views	Separate summaries in the UI, audio and PDF could drift apart.	One shared structured summary per hotspot, used by every view, the assistant and the PDF.
Hindi / Marathi audio	Browser voices for these languages are not installed on every device.	Summary generated in the chosen language; clear message instead of reading it in a wrong voice.
Email trust and security	Fake "sent" messages and credentials in the browser would both be unacceptable.	Server-side SMTP from .env, success shown only after the server accepts, test outbox mode.

SECTION 9

Risks & mitigation strategies

Risk	Mitigation strategy
False alarms mislead decisions	Every event has a replay and the measured reasons; the tool supports human review and never assigns fault. Borderline cases are labelled for review.

Risk	Mitigation strategy
Missed events (detector limits at night / far away)	Larger YOLO11 models, GPU inference, and honest “not assessed” states. Next: night-tuned models and multi-camera fusion.
Privacy and surveillance concerns	Blur faces/heads before storage, delete source videos, keep only anonymous trajectories, aggregate at site level.
Uncalibrated distances	Per-clip perspective model; values shown as approximate; optional one-time camera calibration in future.
Data quality of crash records	Historical and AI-observed evidence are kept separate and shown side by side; the CSV is never modified.
Adoption by authorities	Offline, low-cost, explainable output that fits existing road-safety audit workflows (reports, PDFs).
Model licensing	Licence documented (AGPL-3.0); alternative permissive detectors can be swapped behind the same interface.
Email credentials / misuse	Credentials only in a server-side .env (git-ignored, never in backups); single validated recipient; header injection stripped.
Over-trusting the assistant	It only restates Verge's own evidence, cites its source on every answer, and repeats that Verge is decision support.

SECTION 10

Impact

- **Earlier intervention.** A junction can be flagged from near-miss evidence before the next fatal crash, not after.
- **Better use of limited budgets.** Engineers see a ranked, evidence-backed list instead of anecdotes; the score shows exactly why a site is ranked high.
- **Faster evidence.** Hours of footage become a handful of events with replays and PDFs that can be attached to audit files, tenders or police reports.
- **Protecting vulnerable road users.** Vehicle–pedestrian and two-wheeler conflicts are measured explicitly; pedestrians and two-wheelers make up a large share of road deaths in India.
- **Privacy-respecting smart city.** Uses existing cameras without identifying people.
- **SDG alignment.** SDG 3.6 (halve road-traffic deaths and injuries) and SDG 11.2 (safe transport systems).

SECTION 11

Future implementations

Direction	What it enables
Live CCTV streams	Continuous monitoring of a junction instead of uploaded clips; alerts when conflict rates rise.
Before/after studies	Measure whether a speed breaker, signal change or crossing actually reduced conflicts.
City-wide dashboard	Many cameras and sites, trends over time, heatmaps of conflict points within a junction.
Real MoRTH / state data	Swap the sample CSV for official accident records; add road type and traffic volume exposure.
Emergency response	Crash detection triggers a notification to traffic control or ambulance dispatch.
More behaviours	Red-light running, wrong-way driving, speeding, helmet and seat-belt use, jaywalking.
Better perception	Night-time and rain models, re-identification tracking, multi-camera fusion, one-time calibration.
Mobile / dash-cam app	Citizens or fleet operators report dangerous spots with automatically analysed clips.
Planning integration	Export to GIS tools used by municipal engineers; prioritised work orders for road safety audits.
Natural-language layer	An optional on-premise language model that rephrases the assistant's grounded answers, fed only the structured facts.
Scheduled reporting	Weekly emailed review digests per ward or corridor, generated from the same reports.

SECTION 12

References

[1] Ministry of Road Transport & Highways, Government of India. Road Accidents in India 2022.

[2] World Health Organization. Global Status Report on Road Safety 2023.

[3] Hydén, C. (1987). The development of a method for traffic safety evaluation: The Swedish Traffic Conflicts Technique. Lund University.

[4] Hayward, J. C. (1972). Near-miss determination through use of a scale of danger. Highway Research Record 384.

[5] Allen, B. L., Shin, B. T., Cooper, P. J. (1978). Analysis of traffic conflicts and collisions. Transportation Research Record 667.

[6] Ijjina, E. P., Chand, D., Gupta, S., Goutham, K. (2019). Computer vision-based accident detection in traffic surveillance. ICCCNT 2019.

[7] Jocher, G., Qiu, J. (2024). Ultralytics YOLO11. github.com/ultralytics/ultralytics (AGPL-3.0).

[8] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal; opencv.org.

[9] ONNX Runtime and DirectML execution provider, Microsoft. onnxruntime.ai.

[10] FastAPI (tiangolo.com), Leaflet (leafletjs.com), OpenStreetMap contributors (openstreetmap.org).

[11] United Nations Sustainable Development Goals: targets 3.6 and 11.2. sdgs.un.org.

[12] Project repository documentation: README.md, docs/architecture.md, docs/cv-benchmark.md, models/README.md.